Fictional app, sample report. Not a security certification or penetration test

Northwind Notes: sample code review

Summary for the owner

Northwind Notes is a small fictional notes app with a search box and team-only links. Its screens are understandable, and the code is small enough to repair without replacing the application. The Broken copy nevertheless crosses three important boundaries: who may read a note, how saved text becomes page content, and which fields a request may create.

Within the fictional data set, a Blue team member can open a Red team note just by changing its ID. A saved HTML fragment can also execute a harmless page marker. The create operation accepts empty values and caller-supplied ownership fields. These are the first three repairs to make before treating the pattern as a basis for a real product. There are no real accounts, credentials or customer records in this sample.

Search and feedback need a second pass. An old query can replace a newer result, an unsuccessful request looks like an empty list, and the draft snapshot keeps the initial text. A second filter disagrees with the API about letter case. These problems can make people distrust the app even when the underlying note is intact.

The Fixed copy implements the three High findings and the smaller repairs described below. Paired tests establish the Broken symptoms and the repaired behavior. This is bounded evidence for this sample, not a statement that a future application is ready for release. The identity is still a fixed local fixture, and data still disappears when the process ends.

Scope and method

Reviewed the local Next.js app, its note API, shared in-memory records, browser components and locked production dependency graph. Read both implementations, ran the local build, exercised the API and browser, and inspected the changes against the regression tests. The original runtime snapshot is `967e2fe`; the accompanying tests are separate from the Broken copy.

The API obtains a trusted demo viewer from a fixture. No request parameter can change that viewer in Fixed. This isolates record authorization; it does not implement or assess real sign-in, session integrity, invitations or membership administration. Team-only links carry a note ID, not a permission grant. Deployment configuration, live integrations, operational controls and legal compliance were outside scope.

The sample checks passed: 18 unit contracts, including 3 expected Broken failures, and 18 browser contracts, including 7 expected Broken failures. Separate passing checks establish the exact failures. The browser uses controlled response order and clocks for delayed work. Formatting, lint, type checking and the production build passed. On 4 November 2026, `npm audit --omit=dev` reported zero production advisories for this lockfile. This is a point-in-time dependency check; no dependency advisory finding is asserted here.

Severity reflects the demonstrated pattern if carried into a real app. Effort estimates cover focused code and test work, overlap in places, and are not a project quote. Diff excerpts show actual changed lines; complete paired files are linked.

Findings at a glance

ID	Finding	Severity	Area	Effort
F01	Note IDs bypass team ow nership	High	Authorization	2-4 hours
F02	Saved markup executes in the page	High	Rendering	1-2 hours
F03	Creation trusts unvalidated fields	High	Input boundary	2-3 hours
F04	Search has no debounce or cancellation	Medium	Async state	2-3 hours
F05	A failed request looks like no results	Medium	Feedback	1-2 hours
F06	A stale closure freezes the draft snapshot	Medium	React effects	0.5-1 hour
F07	Duplicate filters disagree about case	Low	Maintainability	0.5-1 hour
F08	Client code contains an unused key pattern	Low	Configuration	0.5 hour
F09	The original app has no regression cases	Medium	Tests	2-4 hours

F01 - Note IDs bypass team ownership

Where: samples/ai-built-review/broken/notes.ts:8. **High.**

Why it matters: Looking up an ID is not an ownership check. Broken returns another team's note and includes it in search. A team-only link does not make that access legitimate.

Reproduce: In Broken, open `red-note` as the displayed Blue member. The API returns 200 and Red team text. In Fixed, the same request returns 403, even with `teamId=team-red` in the query. Search returns only Blue records.

Repair: Check the trusted viewer's membership on every read and list operation; derive ownership on creation. The local viewer remains a fixture.

```
-  read(store, id) {
+  read(store, id, viewer) {
     const note = store.notes.get(id);
     if (!note) throw new NoteError(404, 'Note not found.');
+    if (!viewer.teamIds.includes(note.teamId))
+      throw new NoteError(403, 'This note belongs to another team.');
     return note;
```

Evidence: The shared unit and HTTP/browser access contracts fail on Broken and pass on Fixed. The explicit Broken check confirms the returned Red text.

F02 - Saved markup executes in the page

Where: samples/ai-built-review/broken/note-body.tsx:3. **High.**

Why it matters: Saved note content is treated as HTML, allowing it to change page behavior. The demonstrated payload changes only a local DOM marker; it sends no data anywhere.

Reproduce: Save a note containing an image with a missing local source and an `onerror` handler that sets `document.documentElement.dataset.noteXss`. Open the note or its team link. Broken executes the marker. Fixed displays the complete string as text.

Repair: Notes are plain text, so render them as a React text child. Do not add HTML support without a separate content policy.

```
-    <div className="note-body" data-testid="note-body" dangerouslySetInnerHTML={{ __html: body }} />
+    <div className="note-body" data-testid="note-body">
+      {body}
+    </div>
```

Evidence: Unit rendering checks escaping. A real browser separately proves execution in Broken and the absence of an active image or marker in Fixed.

F03 - Creation trusts unvalidated fields

Where: samples/ai-built-review/broken/notes.ts:14. **High.**

Why it matters: A TypeScript cast does not validate JSON. Object assignment accepts empty or wrongly typed fields and can overwrite the proposed ID or team.

Reproduce: POST an empty title and body, or add `teamId: "team-red"`. Broken accepts the record. Fixed returns 400 and leaves the store unchanged. A caller with no trusted team membership receives 403.

Repair: Accept only title and body strings, trim them, enforce 1-80 and 1-2000 character bounds, reject unknown fields, and assign the ID and team internally. Malformed JSON receives a useful 400 response.

```
-  create(store, payload) {
-    const note = Object.assign({ id: store.nextId(), teamId: 'team-blue' }, payload) as Note;
+  create(store, payload, viewer) {
+    if (!viewer.teamIds.length) throw new NoteError(403, 'Team membership required.');
+    const input = parseNote(payload);
+    const note = { ...input, id: store.nextId(), teamId: viewer.teamIds[0] };
```

Evidence: Nine invalid-input examples, store-integrity checks, malformed JSON and real HTTP requests cover the boundary. The explicit baseline check confirms accepted caller ownership.

F04 - Search has no debounce or cancellation

Where: samples/ai-built-review/broken/notes-client.tsx:18. **Medium.**

Why it matters: Every edit starts work, and older responses can replace newer results. The extra client filter can then make the current query look empty.

Reproduce: Hold the `pine` response, complete `fern`, then release `pine`. Broken loses the Fern result; Fixed retains it. The test controls the response order instead of depending on network speed.

Repair: Wait 250 ms after input, abort obsolete work on cleanup, and guard state updates so only the current request can change results or loading feedback.

```
-    fetch('/api/broken/notes?q=' + encodeURIComponent(query))
+        const response = await fetch('/api/fixed/notes?q=' + encodeURIComponent(query), {
+          signal: controller.signal,
+        });
```

Evidence: Both implementations run the same newest-result contract; an additional passing check proves Broken loses it after the held response settles.

F05 - A failed request looks like no results

Where: samples/ai-built-review/broken/notes-client.tsx:19. **Medium.**

Why it matters: Ignoring HTTP status turns an unavailable query into an empty list. A visitor cannot tell whether to change the query or retry.

Reproduce: Return a controlled 503 for a query. Broken displays "No notes found." Fixed displays an error with Try again. Saving also has pending and failure feedback in Fixed.

Repair: Check status before using the payload, identify the current loading state, catch failures and offer a retry. Avoid replacing an actionable failure with a successful empty state.

```
-      .then((r) => r.json())
+        if (!response.ok) throw Error('Could not load notes. Try again.');
```

Evidence: Paired browser checks cover the failed query, and a separate baseline assertion confirms the misleading empty state.

F06 - A stale closure freezes the draft snapshot

Where: samples/ai-built-review/broken/notes-client.tsx:29. **Medium.**

Why it matters: The mount-only interval keeps the first body value. Later edits never reach the displayed snapshot. This is in-memory draft feedback, not persistent autosave.

Reproduce: Edit Body to "Revised draft" and advance the controlled clock. Broken still shows "First draft"; Fixed shows the edited text.

Repair: Reschedule the snapshot when body changes and cancel the previous timer. The delayed callback now captures the current draft.

```
-    const timer = setInterval(() => setSnapshot(body), 1000);
-    return () => clearInterval(timer);
-  }, []);
+    const timer = setTimeout(() => setSnapshot(body), 1000);
+    return () => clearTimeout(timer);
+  }, [body]);
```

Evidence: Paired clock-controlled tests and a separate check of the captured initial value demonstrate both outcomes.

F07 - Duplicate filters disagree about case

Where: samples/ai-built-review/broken/notes-client.tsx:49, with a second rule at samples/ai-built-review/broken/notes.ts:5. **Low.**

Why it matters: The API matches case-insensitively, then the browser filters the same results case-sensitively. Two copies of a rule silently disagree.

Reproduce: Search for `WELCOME`. The API returns Welcome notes, but Broken removes it. Fixed displays the returned record.

Repair: Keep the search rule at the API boundary and render its results directly. A new client-side filter is unnecessary here.

```
-  const visibleNotes = notes.filter((note) => note.title.includes(query));
-      notes={visibleNotes}
+      notes={notes}
```

Evidence: The same case-insensitive search contract fails as expected on Broken and passes on Fixed; a baseline check confirms the discarded result.

F08 - Client code contains an unused key pattern

Where: samples/ai-built-review/broken/notes-client.tsx:7. **Low.**

Why it matters: The value is exactly `DEMO_KEY_NOT_A_REAL_SECRET`, an obvious harmless placeholder. There is no credential exposure in this sample. Keeping an unused key-shaped setting in browser code nonetheless invites an unsafe substitution later.

Reproduce: Open Broken and inspect its visible connection marker. No integration uses it. Fixed has no marker or key setting.

Repair: Remove the unused setting and prop. If a real integration is later designed, define its trust boundary before adding configuration.

```
-const apiKey = 'DEMO_KEY_NOT_A_REAL_SECRET';
-      marker={apiKey}
```

Evidence: Source comparison shows the removal, and the baseline browser check verifies the marker is only the declared placeholder.

F09 - The original app has no regression cases

Where: samples/ai-built-review/package.json:18 in runtime snapshot `967e2fe`; added coverage is at samples/ai-built-review/fixed/tests/notes.test.tsx:14. **Medium.**

Why it matters: The original runtime snapshot has a test command but no test files in this sample's tree. Working screens alone give no repeatable check of team access, rendering or malformed input. This is a baseline observation, not a claim that the current repository has no tests.

Reproduce: Inspect the sample tree at `967e2fe`: there are no unit or browser test cases. The following commit supplies the companion suite under `fixed/tests/` and `e2e/`, keeping Broken as a negative control.

Repair: Add focused contracts for the important boundaries and run them in CI. An unexpected Broken pass must fail the suite; separate passing tests must establish the actual symptoms.

```
+    const contract = name === 'Broken' ? it.fails : it;
+    contract('rejects another team’s note and excludes it from search', () => {
+      const store = createStore();
+      expect(() => api.read(store, 'red-note', demoViewer)).toThrow(NoteError);
+      expect(api.list(store, '', demoViewer).map((n) => n.id)).not.toContain('red-note');
+    });
```

Evidence: The current 18 unit and 18 browser contracts pass, including the expected failures. A separate CI job runs the local app; publishing the report does not publish its API.

Positive notes

- The sample is small, typed and easy to follow. The data model does not need replacement to make the three main repairs.
- Shared records, view and HTTP error handling keep the comparison focused. Fixed derives ownership from its trusted identity rather than a client field.
- The plain-text content policy is simple to test. No rich-text parser or real credential is needed for the example.
- Controlled clocks and response gates make the asynchronous regressions reproducible. Existing case-library checks remain separate.

Next steps

1. Keep F01-F03 and their unit and real-browser contracts together. Verify reads, search and team-only links whenever ownership rules change.
2. Keep F04-F06 and F09 in the normal check sequence: failure feedback, response order, current drafts and the narrow keyboard flow should remain covered.
3. Retain the single search rule and remove unused configuration patterns as in F07-F08. Avoid reintroducing a second filter to patch an unrelated display issue.
4. Before adapting the sample to a real product, separately design authenticated identity, session lifecycle, membership changes, durable data and concurrent edits. The fixed local identity and process memory do not supply those capabilities.

Run the commands in the sample README. The report, web page and PDF share this same Markdown content.